

STM32F411 General-Purpose Embedded Kernel: 9-Layer Outline

A reusable kernel/runtime architecture for STM32F411 projects, contrasted with a custom Logisim CPU kernel.

Purpose

This is not a full implementation plan for every driver. It is the architectural outline: what layers exist, what each layer owns, how they interact, and how the design differs from a kernel written for a custom Logisim CPU. The target is a reusable STM32F411 embedded runtime that can eventually support networking, an ST7735R/ST7735S-class 128x128 SPI LCD, and security features.

Important hardware constraint: STM32F411 does not have an integrated Ethernet MAC. Ethernet or Wi-Fi needs an external module/controller. Reasonable paths include W5500-style SPI Ethernet, ENC28J60-style SPI Ethernet, or ESP8266/ESP32-style Wi-Fi over UART/SPI. Libraries are fine; the kernel architecture should isolate them behind a network-driver interface.

The 9 layers

Layer	Name	Main responsibility
0	Board physics + debug	Power, clocks source assumption, SWD wiring, reset, boot/debug access
1	Boot + memory contract	Vector table, Reset_Handler, linker script, stacks, .data/.bss
2	Architecture core	Cortex-M exception model, SVC, PendSV, SysTick, privilege, MPU
3	Chip/board support	RCC, GPIO base setup, pin maps, clock tree, board_init
4	Device drivers / HAL	GPIO, UART, SPI, I2C, timers, DMA, EXTI, watchdog, flash
5	Kernel core	Tasks, scheduler, timebase, IPC, queues, ring buffers, memory pools
6	Protection + policy	Fault handling, privilege boundary, MPU regions, syscall validation, logging
7	Middleware services	Networking stack/module interface, display/graphics, files/config, crypto helpers
8	Application/runtime API	Project code, services, update policy, security policy, app task graph

0. Board physics + debug access

- Owns: power source, common ground, ST-Link SWDIO/SWCLK/VREF/GND wiring, reset button behavior, BOOT0 policy, physical pinout, LCD/module voltage compatibility.
- Why it exists: embedded kernels are not abstract until the board is alive. If SWD wiring, power, or voltage reference is wrong, no higher layer matters.
- Logisim contrast: your Logisim machine had idealized power/reset/debug behavior. The STM32 board has real electrical constraints and external debug hardware.

1. Boot + memory contract

- Owns: linker.ld, startup.c, vector table, initial stack pointer, Reset_Handler, .data copy, .bss clear, stack/heap layout.
- Goal: every project boots the same way and reaches kernel_main() or app_main() predictably.

- Logisim contrast: on your custom CPU, you could define where reset starts and how memory is arranged. On STM32, the Cortex-M vector table and ST memory map are imposed on you.

2. Architecture core: Cortex-M kernel substrate

- Owns: exception handlers, SVC syscall entry, PendSV context switch path, SysTick tick source, MSP/PSP split, privileged/unprivileged thread mode, barriers and special registers.
- Goal: isolate the small amount of CPU-specific assembly and exception trickery.
- Logisim contrast: your syscall/privilege semantics were your ISA design. Here, SVC/PendSV/SysTick already exist and your kernel has to use their rules.

3. Chip/board support package

- Owns: board_init(), system clock setup, RCC defaults, LED/button aliases, UART console pins, SPI bus selection for LCD/Ethernet/Wi-Fi modules.
- Goal: app code uses LED0 or SPI_DISPLAY, not random PC13/PA5/PB6 constants everywhere.
- Logisim contrast: your MMIO map was tiny and yours. STM32 has a vendor-defined map plus board-specific pin routing.

4. Device drivers / HAL

- Owns: GPIO, UART, SPI, I2C, timers, DMA, EXTI, watchdog, flash access, optional USB. Drivers expose small stable interfaces and hide register sequences.
- LCD need: ST7735R/ST7735S LCD should be behind display_spi_write(), lcd_reset(), lcd_cmd(), lcd_data(), draw_pixel()/blit(). A library can implement the controller protocol as long as the SPI/GPIO boundary is clean.
- Networking need: W5500/ENC28J60/ESP module driver should be isolated behind netdev_send(), netdev_poll(), netdev_ioctl(), or similar.
- Logisim contrast: your device model was simple enough to call directly. STM32 projects accumulate peripherals fast, so clean drivers matter more.

5. Kernel core

- Owns: task control blocks, cooperative scheduler first, preemptive scheduler later, sleep queue, monotonic time, queues, event flags, ring buffers, fixed-size memory pools.
- Recommended progression: superloop -> cooperative tasks -> SysTick sleeps -> interrupt-driven queues -> PendSV preemption.
- Logisim contrast: the Logisim kernel was probably dominated by syscalls, terminal service, and memory map constraints. The STM32 kernel is dominated by concurrency with interrupts and devices.

6. Protection + policy layer

- Owns: fault handlers, panic system, assertions, MPU layout, unprivileged task mode, syscall validation, interrupt priority policy, resource permissions, logging redaction.
- Security features: MPU mark SRAM stack regions, no-execute where possible, restrict task access to peripherals, validate SVC arguments, protect kernel data, watchdog recovery, signed-update hooks later.
- The STM32F411 Cortex-M4 includes an MPU according to the datasheet; ST also documents MPU use for protecting critical data and marking SRAM non-executable in application notes.
- Logisim contrast: you designed privilege separation directly in hardware/software. On STM32, the MPU and privilege model exist but are limited and region-based; you must design around those limits.

7. Middleware services

- Owns: TCP/IP integration or module protocol, display stack, graphics primitives, command shell, config storage, crypto wrappers, telemetry protocol.
- Networking paths: use lwIP only if you have a suitable Ethernet/Wi-Fi driver and RAM budget; use a W5500 socket-driver model for simpler Ethernet; use ESP AT-command firmware for easier Wi-Fi at the cost of module dependence.
- Display path: ST7735R/ST7735S over SPI with a framebuffer if RAM allows, or line/tile rendering if RAM is tight. 128x128 RGB565 framebuffer is $128 \times 128 \times 2 = 32768$ bytes, which is plausible in 128 KB SRAM but not free.
- Logisim contrast: your terminal was tiny and custom. Here, display/network middleware can dwarf the kernel if boundaries are not enforced.

8. Application/runtime API

- Owns: project-specific tasks, service graph, network policy, display UI, command protocol, security mode, update behavior.
- Goal: future projects import the kernel and board package, then define tasks/services rather than rewriting startup, drivers, queues, and fault handling.
- Logisim contrast: your user program and kernel lived in a tiny custom universe. On STM32, the application is statically linked into the firmware unless you intentionally build a loader/MPU-isolated task model.

Recommended repository shape

```
stm32-kernel/  
|-- arch/  
|   |-- cortex_m4/  
|   |   |-- startup.c  
|   |   |-- vector_table.c  
|   |   |-- exception.c  
|   |   |-- svc.c  
|   |   |-- pendsv.S  
|   |   |-- mpu.c  
|-- chips/  
|   |-- stm32f411/  
|   |   |-- rcc.c  
|   |   |-- gpio.c  
|   |   |-- uart.c  
|   |   |-- spi.c  
|   |   |-- timer.c  
|   |   |-- linker.ld  
|-- boards/  
|   |-- f411_blackpill/  
|   |   |-- board.c  
|   |   |-- board.h  
|   |   |-- pins.h  
|-- kernel/  
|   |-- task.c  
|   |-- sched.c  
|   |-- time.c  
|   |-- queue.c  
|   |-- event.c  
|   |-- ringbuf.c  
|   |-- pool.c  
|   |-- syscall.c  
|   |-- panic.c  
|-- services/  
|   |-- display/  
|   |   |-- st7735.c  
|   |   |-- gfx.c  
|   |-- net/  
|   |   |-- netdev.h  
|   |   |-- w5500.c  
|   |   |-- esp_at.c  
|   |-- lwip_port/      # optional later
```

```

|  |-- security/
|  |  |-- auth.c
|  |  |-- secure_config.c
|  |  |-- update_verify.c
|  |-- apps/
|  |  |-- blink/
|  |  |-- lcd_demo/
|  |  |-- net_console/

```

Kernel evolution roadmap

Phase	Build this	Stop when this works
1	Boot/runtime + ST-Link debug	Break at Reset_Handler and main; blink PC13.
2	Board package + GPIO/UART	app_main() can call board_led_toggle() and kprintf().
3	SysTick timebase	k_millis() and k_sleep_ms() replace fake delay loops.
4	Cooperative scheduler	Two tasks blink/log without one giant main loop.
5	Interrupt-safe queues	UART RX ISR pushes bytes; task consumes bytes.
6	SPI display service	ST7735 init + clear screen + draw text/rectangles.
7	Network service	External W5500 or ESP module can send/receive packets/messages.
8	Protection boundary	SVC gate, unprivileged task mode, MPU regions, fault dumps.
9	Hardening + reuse	watchdog, asserts, config policy, update hooks, documented APIs.

Security outline

Security should not be pasted on at the end. It should show up as boundaries in the architecture. For this class of MCU, security is not Linux-level process isolation; it is controlled privilege, strict driver boundaries, robust parsing, fault containment, and recovery.

- Run the kernel privileged; run selected application tasks unprivileged using PSP.
- Expose privileged services through SVC rather than direct peripheral access.
- Use the MPU to protect kernel data, task stacks, peripheral regions, and no-execute RAM regions where practical.
- Validate all pointers/lengths passed through syscalls or message queues.
- Use fixed-size pools/ring buffers to reduce heap corruption risk.
- Add stack canaries/high-water checks for every task stack.
- Treat network packets, Wi-Fi module messages, and display command inputs as untrusted.
- Use watchdog recovery and persistent crash reason storage.
- Plan signed update verification only after the basic bootloader/story is clear.

Networking and LCD placement

The LCD and networking should not invade the kernel core. The kernel core should know about tasks, time, queues, and memory. The display service should know about ST7735 commands and graphics. The network service should know about packets or module messages. The HAL should know about SPI/UART/GPIO registers.

```

Application task
|
v
Display API: gfx_draw_text(), gfx_blit(), lcd_present()
|
v

```

```

ST7735 driver: init sequence, command/data writes, address window
|
v
SPI HAL + GPIO HAL: SPI write, CS/DC/RST pins
|
v
STM32 registers

Application/network service
|
v
socket/message API
|
v
network stack or module driver: W5500 / ESP AT / lwIP port
|
v
SPI or UART HAL
|
v
STM32 registers

```

Clean contrast with the Logisim kernel

Topic	Logisim CPU kernel	STM32F411 kernel/runtime
Machine contract	You invented ISA, memory map, syscall rules, privilege levels.	ARMv7-M must define reset, exceptions, memory map, peripheral registers.
Boot	Could start wherever your ROM/CPU design said.	Must satisfy Cortex-M vector table and startup rules.
Syscalls	Custom instruction/protocol by your ISA.	Use SVC exception and stacked register context.
Scheduling	Whatever you implement on your CPU.	Use SysTick for time, PendSV for context switch, NVIC priorities.
Memory protection	Designed into your CPU/kernel model.	MPU region model; useful but not VM/process isolation.
Devices	Tiny custom MMIO devices.	Many vendor peripherals plus external modules.
Debug	Your simulator/assembler/disassembly world.	ST-Link/OpenOCD/GDB live register and memory inspection.
Reuse target	Reusable inside your custom CPU ecosystem.	Reusable across Cortex-M/STM32 projects if layered cleanly.

Design rule

Do not build a mega-framework first. Build a bootable runtime, then one driver, then time, then UART, then scheduler, then queues, then LCD, then network, then security boundaries. Generality should be extracted from real repeated use, not guessed in advance.

Good early API:

```

gpio_write(LED0, 1);
uart_write(CONSOLE, "boot
");
k_sleep_ms(100);
queue_send(&q, &msg);

```

Bad early API:

```

universal_device_manager_register_bus_peripheral_ex(...)

```

Sources

1. STM32F411xC/xE datasheet: <https://www.st.com/resource/en/datasheet/stm32f411ce.pdf>
2. RM0383 STM32F411xC/E reference manual: https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xce-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
3. PM0214 STM32 Cortex-M4 programming manual: https://www.st.com/resource/en/programming_manual/pm0214-stm32-cortexm4-mcus-and-mpus-programming-manual-stmicroelectronics.pdf
4. OpenOCD user guide: <https://openocd.org/doc/pdf/openocd.pdf>

5. AN4838 STM32 MPU management: https://www.st.com/resource/en/application_note/an4838-introduction-to-memory-protection-unit-management-on-stm32-mcus-stmicroelectronics.pdf

6. 1.44 inch ST7735S 128x128 SPI module reference: https://www.lcdwiki.com/1.44inch_SPI_Module_ST7735S_SKU:MSP1443

These sources are used for the hardware facts and layer choices: STM32F411 memory/peripheral facts, Cortex-M4 SVC/PendSV/SysTick/MPU behavior, OpenOCD/ST-Link flashing/debugging, and ST7735-class SPI display assumptions.